

Capstone Project by CA. Rajat Bhatia

AICA - Level 2 - Batch 85

Prompt File

Build: University MOU & Agreement Repository

Build a simple, complete and deployable web application for a university to maintain a centralized repository of all **MOUs, agreements and contracts**.

The application should have a professional university-administration interface and include a frontend, backend, database, authentication and secure document storage. Keep the architecture simple and practical. Do not over-engineer it.

Users

Create two roles:

Admin

- * Manage users and departments.
- * View and manage all agreements.
- * View dashboard, reports and activity history.

Department User

- * Access only their department's agreements.
- * Upload and add agreements.
- * Edit, search, view and download their records.

Main Navigation

Create these sections:

- * Dashboard
- * Agreements
- * Add Agreement
- * Expiring Soon
- * Reports
- * Departments (Admin)
- * Users (Admin)
- * Activity Log (Admin)

Dashboard

Show:

- * Total Agreements
- * Active
- * Expired
- * Expiring Soon
- * Financial Agreements
- * Number of Departments

Also show recent agreements and agreements approaching expiry.

Agreement Repository

Create a searchable and filterable table of all agreements.

Store these basic fields:

- * Reference/Agreement Number
- * Title
- * Type: MOU, Agreement, Contract, MoA, Partnership or Other
- * Department
- * First Party
- * Second Party
- * Purpose
- * Short Summary
- * Signing Date
- * Start Date
- * Expiry Date
- * Status
- * Financial Implication: Yes / No / Not Clear
- * Financial Amount, if mentioned
- * Important Remarks
- * Original Document

Allow search/filter by party, department, type, status and dates.

Add Agreement & Document Reading

The user should be able to upload a PDF or DOCX agreement.

After upload, automatically read the document and try to extract:

- * First Party
- * Second Party
- * Agreement Type
- * Purpose
- * Signing/Start Date
- * Expiry Date
- * Financial information
- * Important clauses
- * Short summary

Show the extracted information in an editable form.

The user must review and correct the information before saving.

If automatic extraction does not work, the user must still be able to enter all information manually.

Store the original document with the agreement and allow viewing/downloading later.

Clause Suggestions

On the agreement details page, include a **"Clause Suggestions"** section.

After reading an agreement, identify common areas that may be missing or unclear, such as:

- * Confidentiality
- * Intellectual Property
- * Data Protection
- * Termination
- * Renewal
- * Dispute Resolution
- * Governing Law
- * Liability/Indemnity
- * Force Majeure
- * Payment/Financial Terms

Give a short general suggestion for consideration.

Clearly state that these suggestions are **general informational guidance and not legal advice** and should be reviewed by the university's authorized/legal authority.

Agreement Details

When a record is opened, show:

****Overview | Parties | Financial Information | Important Clauses | Document | Suggestions | History****

Allow editing and document download.

Expiry Tracking

Automatically calculate status from the expiry date:

- * Active

- * Expiring Soon

- * Expired

- * No Expiry Date

Create an ****Expiring Soon**** page showing agreements expiring within the next 90 days.

Reports

Provide simple reports for:

- * Agreements by Department

- * Agreements by Type

- * Active vs Expired

- * Expiring Soon

- * Financial vs Non-Financial

- * Agreements by Year

Allow CSV/Excel export if straightforward to implement.

Admin

Admin can create, edit and deactivate departments and users and assign users to departments.

Department users must not be able to access another department's records.

Maintain a simple activity log for uploads, edits and deletions.

Design

Use a clean, modern and professional university administration design.

Use a simple sidebar, dashboard cards, tables, search, filters and status badges.

Make it responsive and easy for non-technical staff to use.

Critical Requirement

Build a ****working application, not a UI demonstration****.

The following complete workflow must work:

****Login → Dashboard → Add Agreement → Upload Document → Extract Information → Review/Edit → Save → Database → Search/Filter → View Agreement → Download Document****

All major buttons, forms, navigation and database operations must work.

The core repository must continue working even if AI document analysis is unavailable. AI should enhance document extraction, summarization and clause suggestions, but uploading, editing, searching, filtering, viewing, downloading and reporting must continue to work independently.

Use a simple, reliable technology stack and database supported by Google AI Studio. Keep the project reasonably small so it can be easily maintained and deployed.

Include basic sample agreements/data for testing, clearly marked as demo data.

Before completing the project, test the major workflows and fix obvious errors, broken navigation, forms and database operations.

The goal is a ****small but genuinely usable University MOU & Agreement Repository****, not an unnecessarily large enterprise system.